

RTOS Project:

Part 1 – Non-preemptive Scheduler

Total Marks: (25% total)

Due: Thursday Week 12 Lab session (submit, explain, and present code at end of the session)

Useful Resources

Lecture notes from the course website.

Online lectures from the course website.

Information

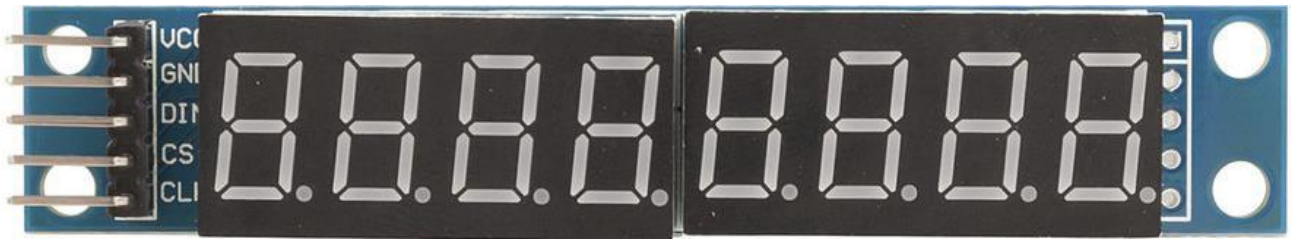
- This module is due on **Week 12**. A copy of the lab and presentation will be collected at the end of the lab session. You will demonstrate and present your code in week 12.
- A template is provided. It can be compiled using the VS code environment and requires a RPi Pico Board and a SPI based 8 x 7 segment display board.
- If the completed code cannot be compiled and downloaded then it will be deemed as not running.
- You must submit the preliminary before the **2nd** lab (Week 8) and a copy of your current code at the submission point each week.
- The final copy will be submitted at the end of the Week 12 Laboratory session.
- You will **not** need to write the `save_context()` and `load_context()` functions or the `systick_Handler()` ISR. The `systick_Handler()` saves the context, calls the `Scheduler()` function, and then loads the context. I have written the `systick_Handler()` function as it is quite complex. You only need to write the scheduler function.

Project

The project is broken up into separate parts:

1. A non-preemptive round robin scheduler based RTOS (20 marks)
2. A pre-emptive round robin scheduler based RTOS. Preemption comes from the scheduler not an external interrupt. (5 marks)
3. IEEE format report (5 marks)

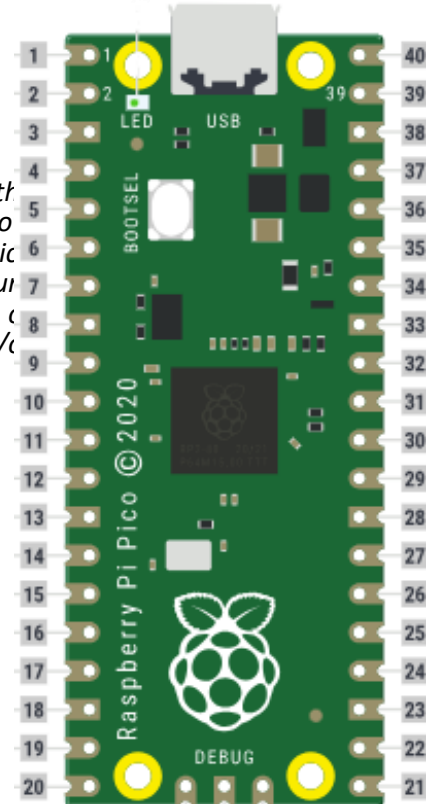
Introduction



Hardware Connections

You will need to connect the supplied 8 x 7segment display to the Raspberry Pi pico. Look at the supplied data sheets and the figure below for the connections of the display. The display is common cathode which means that you must write a 0 to the pins that you want to be able to turn on. Care must be taken to ensure that the correct pins are connected or the display will not work with the possibility of damaging the pico and/or the display.

Name	Display	Pico
3.3V	VCC	Pin 39 (VSYS)
Ground	GND	Pin 38 (GND)
Chip Select	CS	Pin 22 (SPIO CS)
Clock	CLK	Pin 24 (SPIO SCK)
Data In	DIN	Pin 25 (SPIO Tx)



Hardware Abstraction Layer

The Hardware Abstraction Layer (HAL) is used to communicate with the seven segment display board (8 displays in total). In the HAL each individual display is represented by a single byte. Therefore, the display can be used as though it were an array of unsigned characters which is called `display_mem`.

Note: Byte 0 is the LHS display (closest to the header) and byte 7 is the RHS display (furthest from the header)

Software

A template is supplied. You will gradually add .c files to the program and fill them in. These functions will replace the ones given in the library. If your function is correct then you should still see the RTOS operating correctly.

The supplied functions for the RTOS are given in Appendix A. Information about the SysTick use and registers are given in Appendix B. The pinout diagram for the pico are given in Appendix C.

Preparatory Questions

To complete this laboratory effectively, you will need to gain an understanding of the task structure (TCB) used in template. The preparatory questions are designed to familiarise you with the task structure and stack format. You will need to discuss the preparatory with your demonstrator before writing the `Load_Task()` function

Preparatory Question 1

The 8 x (seven segment displays) used in this task need to be connected to the pico. (See the introduction to experiment). The data sheet for the display (XC3714) and the pico pinout is supplied with this laboratory script.

You are required to create a table and then write a line of code for an array containing the seven segment display values to represent hexadecimal digits 0 to F, a Decimal Point, and a blank display. Also write a line of code showing how you would access the 'E' digit from the array.

Background

On the board used in the lab, each individual display is represented by 8 segments (7 for the bars, 1 for the decimal point). Each segment in a display can be turned on/off individually (see figure 1). This means that each individual display can be represented by 8 bits (a to g for the bars and DP for the decimal point) according to Table 2. Therefore, you must set the appropriate bit to **1** to turn on the segment and set the bit to a **0** to turn off the segment. (see supplied datasheet for more information)

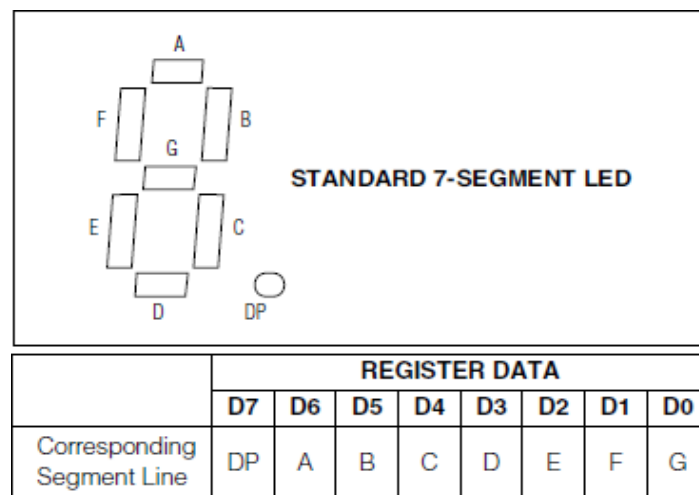


Figure 1: Representation of a single 7 segment display showing common anode and common cathode versions.



Figure 2: Hexadecimal digits generated on a 7 segment display. The red lines are the segments turned on.

Preparatory Question 2

The scheduler's task list consists of a linked list with each list node being a Task Control Block (TCB). Write the functions for a linked list that you will use when adding tasks, removing tasks, and determining the next task to run. (see the requirements below)

Non-Preemptive Round-Robin RTOS Specification

Overview

You are required to implement a **non-preemptive, round-robin RTOS**. Tasks will voluntarily yield control, and the scheduler will cycle through all runnable tasks in a fixed order.

Scheduler

The RTOS must implement a **round-robin scheduler** that iterates through tasks in sequence, giving each task its time slice before moving to the next.

Task List Structure

- The scheduler's task list must be implemented as a **singly linked list**.
- You must provide functions to:
 - **Add** a node (task) to the list
 - **Remove** a node from the list
 - **Search** the list for a specific node
 - **Track the current node** during iteration

Task Identification

- Each task must have a **unique task ID**.
- When a task is added to the scheduler, the RTOS must automatically assign it a unique ID.
 - A simple **incrementing ID counter** is recommended.
- Task IDs are used for task removal.
- Users can view task IDs by running the `ps` command in the shell, which prints the current task list.
-

Time Slicer : Quantum

- The time slice (quantum) is generated using **SysTick**.
-

SysTick Configuration

- The SysTick configuration function must accept the **desired interrupt frequency** as an argument and configure SysTick accordingly.

Output Devices

- The RTOS must drive:
 - The **onboard LED**
 - The **8×7-segment display** (in segment-display mode)
 - The serial terminal

The Shell

A basic command-line shell must be implemented using the serial port. The shell acts as a simple command interpreter that allows the user to interact with RTOS services.

Init Task

The **first task added** to the scheduler must be the **init task**. Its purpose is to initialise the RTOS and then remove itself once initialisation is complete.

The init task responsibilities are to :

- the **HAL**
- Clear the 7-segment display and write **"Hello"**
- Configure the SysTick frequency
- Launch the terminal shell by adding the shell task to the scheduler
- Remove itself from the scheduler once initialisation is complete

Shell Commands

All shell commands:

- Are terminated by a **newline**
- Use **spaces** to separate arguments

Shell Command List

taskname <priority> <arguments>

Adds the specified task to the scheduler. If the task uses the 7-segment display, the argument will specify the display position (0–7).

pt — **Print Task List**

Displays all active tasks, including:

- Task ID
- Task name
- Priority

rt <id> — **Remove Task**

Removes the task with the specified ID from the scheduler.

sd — **Shut Down RTOS**

Shuts down the RTOS by:

- Turning off the display
- Enters an infinite wait loop Only a **reset** will restart the system.

Tasks

Task: Blinky

Argument: Number of LED flashes

Description:

- Flashes the Raspberry Pi Pico onboard LED the specified number of times. Each flash consists of:
- LED ON for 2 seconds
- LED OFF for 2 seconds
- This results in a flash period of 4 seconds. After the requested number of flashes has been completed, the LED is turned OFF and remains OFF.

Task: Counter

Argument: Starting display digit position

Description:

- Continuously counts from **00 to 99** and then repeats from **00**.
- The count value is updated every 0.5 seconds.
- The tens digit is displayed at the specified display position.
- The units digit is displayed at the next display position (digit position + 1).

Task: Flasher

Argument: Display digit position

Description:

- Continuously flashes the decimal point (DP) on the specified display position at a frequency of **2 Hz**.
- DP ON for 0.25 seconds
- DP OFF for 0.25 seconds
- The flashing operation repeats indefinitely.

Task: Hexer

Argument: Display digit position

Description:

- Continuously displays a hexadecimal count on the specified display position.
- The sequence is:
- $F \rightarrow E \rightarrow D \rightarrow \dots \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow F$
- The displayed value changes once per second.
- After reaching F while counting up, the cycle repeats indefinitely.

Task: Splat

Argument: Display digit position

Description:

- Sequentially illuminates the segments of the specified display digit, one segment at a time, until all seven segments and the decimal point are illuminated.
- Once all segments are ON:
- The segments are turned OFF one at a time.
- The order of deactivation is the reverse of the activation sequence.
- The display state changes every **0.25 seconds**, and the pattern repeats continuously.

Example Sequence:

$A \rightarrow A+B \rightarrow A+B+C \rightarrow \dots \rightarrow A+B+C+D+E+F+G+DP$

then

$A+B+C+D+E+F+G \rightarrow A+B+C+D+E+F \rightarrow \dots \rightarrow OFF$

Task: cmdShell

(See the above description of the shell)

Appendix A Supplied Functions

Hardware Abstraction Layer (HAL)

unsigned char * Hardware_Abstraction_Layer(void)

- Library Function
- This function is used to update the display of the seven segment display board used in the laboratory.
- The function returns a pointer to the start of an array of 8 unsigned character. Each unsigned character corresponds to one of the 7 segment displays. This means that the program only needs to write the value to the appropriate element in the array to update the desired display.
- Make sure that the seven segment display board is connected to the pins of the RPi Pico before using the function.

System Timer Functions

uint32_t configure_Systick(uint32_t Timer_Value)

- You write
- This function is used to configure the systick timer to interrupt at the rate determined by the Timer_Value. In this case it is 1MHz.
- Note that the System clock is running at 125MHz/

void isr_systick(void)

- You write by adding to this handler
- This interrupt handler is used to generate the system clock ticks (uint32_t Ticks) and to trigger the context switch using the CONTEXT_SWITCH() ; macro.

void rtos_Start(void)

- This function is prewritten and launches the first task in the scheduler.
- This is performed once after the quatum, and scheduler have been set up and the scheduler has at least 1 task in the Task_List.

RTOS functions

void scheduler(void)

- This function is invoked by the Systick Interrupt which runs at 1MHz. The scheduler should be invoked once every 100 microseconds (Use the system Ticks).
- The role of this function is to search the scheduler's task list for the next task to run and set "CurrentTCB" to point to the address of the TCB for the task to be run. In the round robin scheduler, once Current_TCB has reached the end of the Task_List then it starts again from the beginning of the list.
- The system tick counter also forms part of this function.
- The Task_List variable holds the address of the start of the linked list. The linked list consists of Task Control Block (TCB) with one TCB for each task. If there are no elements in the list then this variable points to NULL.

int add_Task(void (*Task)(int), uint32_t arg, const char *name, uint8_t priority);

- You write
- This function adds a task to the scheduler by placing it in an empty position in the scheduler's task list.
- A Dummy stack frame is required for a each new start.
- For the TCB structure of the new task to be added: (See Appendix B)
 - The id is set to the Task_ID. This will be automatically generated using a counter that is incremented

for each new task added to the scheduler.

- The Task_ID is an arbitrary non-zero/non-negative value.
 - All elements of the stack are initialised for a new task according to Appendix B.
 - The PC register position is set to the function pointer (*Task). This will be a pointer to the task function to add.
 - The argument for the Task (*Task)(int) is stored in the R0 register position
 - The Stack_Pointer is set to the stack position immediately at the bottom of the stack frame.
- The priority level and meaning is determined by the programmer
 - If the task was able to be added the function return the Task Id otherwise it returns a -1

int remove_Task(uint32_t Task_Id)

- You write.
- This function removes the task with an id of Task_id from the scheduler creating an empty position in the tasklist. For the taskinfo structure at this position:
- The id is set to -1. The task_id is an arbitrary non-zero/non-negative value
- All elements of the stack are set to 0.
- The SP is set to 0.
- The priority is set to 0.
- If the task was able to be removed the function return the Task Id otherwise it returns a 0.

void Task(int arg);

- These functions are the user defined functions. The functions should not return and will keep executing by an infinite loop which will prevent the function from completing.

void cmdShell(int dummy)

- You write.
- This task is a function which is the command line interface (CLI) allowing you to interact with RTOS tasks.
- It writes and reads from the serial port. The serial port must already be initiated before this task is run. The main() initiates the usb serial connection as the first set of operations performed.
- TeraTerm or equivalent should be running as the serial monitor program on the PC.
- The dummy parameter can be ignored in your code.

int main(void)

- Prewritten with the source code available in the main.c file.
- The main() does the following:
 - Initialises and initiates the USB – Serial Connections
 - Adds the initd task to the scheduler. The purpose of the initd task is to start the HAL and launch other tasks. Specifically, it launches the cmdShell() task.
 - The main configures the systick and starts the scheduler.

MACROs and Helper Functions

DISABLE_INT(); and ENABLE_INT();

- These macros are found in RTOS.h. They enable and disable the interrupts.

CONTEXT_SWITCH();

- This macros is found in RTOS.h. It triggers the context switch which saves the context, calls the scheduler, and then loads the new context.

void delay(uint32_t ms);

- This function delays for the given number of milliseconds.

- It is provided as part of the library for you to use but you are expected to replace it later. It is provided so that you can delay actions before implementing the system Tick.

void readln(char *line, int size);

- This function reads a string from the serial terminal until an enter is pressed or the size of the string is exceeded. Backspace capability has been added.
- It is provided as part of the library for you to use in your CLI.

Appendix B The Dummy Stack Frame

Background

When a timer interrupt occurs the task's context will be pushed onto the stack. The PSR, PC, LR, R12, R3, R2, R1 registers are automatically pushed onto the stack. Then registers R4 – R11 are also pushed onto the stack. Therefore it is important to set the stack to the right starting values before it is first run. The position of the registers in the stack (treated like an array) are as follows:

Register Name	Initial Value	Index into the stack array	Comments
PSR	0x01000000	0	sets 'T' to 1 (thumb mode)
PC	Address of Task to be run	-1	Program Counter (R15)
LR	Don't care	-2	Link Register (R14)
R12	Don't care	-3	Scratch Register
R3	Don't care	-4	4th argument to function
R2	Don't care	-5	3rd argument to function
R1	Don't care	-6	2nd argument to function
R0	uint32_t argument	-7	1st argument to function / return value
R7	Don't care	-8	General Register
R6	Don't care	-9	General Register
R5	Don't care	-10	General Register
R4	Don't care	-11	General Register
R11	Don't care	-12	Link Register
R10	Don't care	-13	Link Register
R9	Don't care	-14	Link Register
R8		-15	Stack Pointer in the TCB should point here
Empty	Empty		Next Stack Frame

Table 1: Representation of the context in the stack array for a task that has been added but not run. Each register is at a unique position in the array and therefore can be accessed using the stack array via its index in the array. When you add a new task to the scheduler you must write the PSR and the PC to the correct place in the stack and you must set the Stack Pointer in the TCB to point to the address of R8 in the stack frame.

Note:

The stack pointer element in the TCB must be pointing to the array element R8 as shown in the pink text.

Hint: The stack is a Top Down stack which means that it starts at the higher memory address and works towards the lower memory address. Therefore the PSR element is at the end of the array.

Appendix C SysTick Information

The ARM GPIO is well documented since it is a fundamental component of the ARM microcontroller design.

The hardware systick can be accessed using the following registers from the stdlib.h include file as shown below:

```
#include "pico/stdlib.h"
```

The led on the rpi pico is on GPIO pin 25.

```
systick_hw->csr    // control status register
systick_hw->rvr    // 24 bit reload value register : determines the frequency of interrupts
systick_hw->cvr    // current value register
```

Note that the system clock is 125MHz. At 125MHz it would overflow every 0.134 seconds. The

register details are given below. (From RP2040 Datasheet – Rpi Foundation)

MOPLUS: SYST_CSR Register

Offset: 0xe010

Description

Use the SysTick Control and Status Register to enable the SysTick features.

Bits	Name	Description	Type	Reset
31:17	Reserved.	-	-	-
16	COUNTFLAG	Returns 1 if timer counted to 0 since last time this was read. Clears on read by application or debugger.	RO	0x0
15:3	Reserved.	-	-	-
2	CLKSOURCE	SysTick clock source. Always reads as one if SYST_CALIB reports NOREF. Selects the SysTick timer clock source: 0 = External reference clock. 1 = Processor clock.	RW	0x0
1	TICKINT	Enables SysTick exception request: 0 = Counting down to zero does not assert the SysTick exception request. 1 = Counting down to zero to asserts the SysTick exception request.	RW	0x0
0	ENABLE	Enable SysTick counter: 0 = Counter disabled. 1 = Counter enabled.	RW	0x0

M0PLUS: SYST_RVR Register

Offset: 0xe014

Description

Use the SysTick Reload Value Register to specify the start value to load into the current value register when the counter reaches 0. It can be any value between 0 and 0x00FFFFFF. A start value of 0 is possible, but has no effect because the SysTick interrupt and COUNTFLAG are activated when counting from 1 to 0. The reset value of this register is UNKNOWN.

To generate a multi-shot timer with a period of N processor clock cycles, use a RELOAD value of N-1. For example, if the SysTick interrupt is required every 100 clock pulses, set RELOAD to 99.

Bits	Name	Description	Type	Reset
31:24	Reserved.	-	-	-
23:0	RELOAD	Value to load into the SysTick Current Value Register when the counter reaches 0.	RW	0x000000

M0PLUS: SYST_CVR Register

Offset: 0xe018

Description

Use the SysTick Current Value Register to find the current value in the register. The reset value of this register is UNKNOWN.

Bits	Name	Description	Type	Reset
31:24	Reserved.	-	-	-
23:0	CURRENT	Reads return the current value of the SysTick counter. This register is write-clear. Writing to it with any value clears the register to 0. Clearing this register also clears the COUNTFLAG bit of the SysTick Control and Status Register.	RW	0x000000

The mode of operation is as follows:

- 1) Disable the systick timer
- 2) Calculate and set the value to give the desired frequency of interrupts
- 3) Clear the current counter
- 4) Enable the systick timer and specify the course source

In this lab you will set the systick timer to :

- be a count down timer and interrupt when the counter = 0.
- use the processor clock as the clock source. (125MHz)

Appendix D **GPIO Information**

The ARM GPIO is well documented since it is a fundamental component of the ARM microcontroller design.

The hardware systick can be accessed using the following functions from the stdlib.h include file as shown below:

```
#include "pico/stdlib.h"
```

The led on the rpi pico is on GPIO pin 25.

Function	Description
gpio_init(GPIO_PIN)	Initialise the GPIO subsystem for the designated gpio pin.
gpio_set_dir(GPIO_PIN, DIR)	Sets the pin direction (GPIO_OUT or GPIO_IN)
gpio_put(GPIO_PIN, value)	Sets the pin to a 1 or 0 usin (true or false)

fl

Appendix E RPi Pico Pinout

